画像処理実験 第 1 回

学生番号: 09B24490
加藤弘都 (KATO, HIROTO)

2026 年 10 月 6 日

1　課題 2

1.1　[2-1]

1.1.1　第 1 回の「適当な処理 7」と afn の比較，afn2, afn3

まず，第 1 回の「適当な処理 7」と比較する．比較したとき，[2.1 geom] は回転角度，拡大倍率，c，s を main 関数で定義していて，afn 関数では u，v と新しい画像を定義している．両者の処理に違いはなく，定義されている関数が異なるだけだと考える．そのため，両者とも射影変換を用いて 45 度回転させて，3.5 倍に拡大している．

次に afn2 関数について説明する．この関数では，afn 関数と同じ処理をしている．afn 関数では main 関数内の定義より画像の (550,220) 座標を基準に [-150:150,-150:150] の範囲の画像を指定していた．しかし，afn2 関数では，main 関数内で a に指定した行列をかけて，a で指定していた (550,220) 座標から (-550，-220) の平行移動をさせて，(0,0) を基準に画像を指定しようとしている．これにより afn2 関数内で [70:370,400:700] のように画像の範囲を直接指定して拡大・回転させている．

最後に afn3 関数では黒の背景部分を出力せず，画像のみ画面に出力するようにしている．u，v が画像の中にあるときに True，そうでないときに False を格納する配列 m を定義し，m==True のときのみ im に格納している．

1.1.2　射影変換の関数の実装

afn3 関数をもとに以下のような射影変換の関数 proj を実装した．afn3 では z を考えていなかったため z_を求め，u=x_/z_，v=y_/z_に変更する．

```
 1: def proj(dw,dh, src, a):
 2:
 3:
 4:
 5:  im=np.zeros((dh,dw,3),'f')
 6:
 7:  y ,x =np.mgrid[0:dh,0:dw]                    # (i)
 8:
 9:  # z_を追加-----
10:  x_,y_,z_= a[0,0]*x+a[0,1]*y+a[0,2], \
11:          a[1,0]*x+a[1,1]*y+a[1,2], \
12:          a[2,0]*x+a[2,1]*y+a[2,2]
13:
14:  # z_を追加-----
```

```
15:  u,v=x_ / z_, \
16:     y_ / z_
17:
18:  m = (0<=u)&(u<src.shape[1])&(0<=v)&(v<src.shape[0]) # (iv)
19:  # 画像内の座標しか参照しないため，画像を拡大できる
20:
21:  im[y[m],x[m]]= src[v[m].astype('i'),
22:                     u[m].astype('i')]        # (iii) int 型でやっている
23:
24:  return im
```

1.1.3 変換行列の変化による画像の相違

変換行列を変更したとき，図 1 のようになる．左側は変更前で，右側は変更後である．

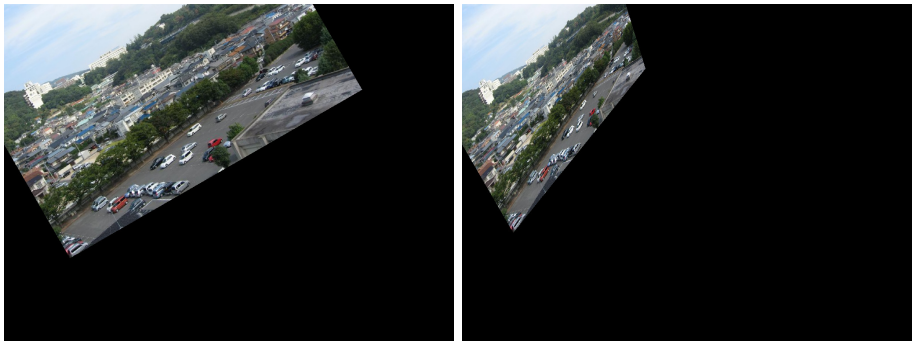


図 1　変換行列の変化による 2 つの画像（左は変更前で右は変更後）

2 つの画像を比較すると，変更前の画像の方が見える画像の面積が広く，画像が見える角度が変化していることがわかる．

1.1.4 回転・拡大/縮小行列 A の一般形

m 倍に拡大し，t 度回転させるとき，「原画像の座標 (u,v) を出力画像の座標 (x,y) に変換する回転・拡大/縮小行列」A の一般形は以下のようになる．

$$A = \begin{pmatrix} \frac{\cos t}{m} & -\frac{\sin t}{m} & 0 \\ \frac{\sin t}{m} & \frac{\cos t}{m} & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

まず点 (p,q) を中心とする m 倍の拡大縮小を行う．以下のように a を定義するとき，t=0，p=0，q=0 という条件のときの m が 1 と 0.5 の違いを図 2 に示す．図のように m の値に応じて画像が拡大・縮小していることがわかる．

```
1:   a=np.array([[ c, -s, q],
2:                [ s,  c, p],
3:                [ 0,  0,  1]])
```

次に，点 (p,q) を中心とする角度 t の回転を行う．m=1，p=0，q=0 という条件のときの t が 0 と $\frac{\pi}{4}$ の違いを図 3 に示す．図のように t の値によって画像が $\frac{\pi}{4}$ だけ回転していることがわかる．

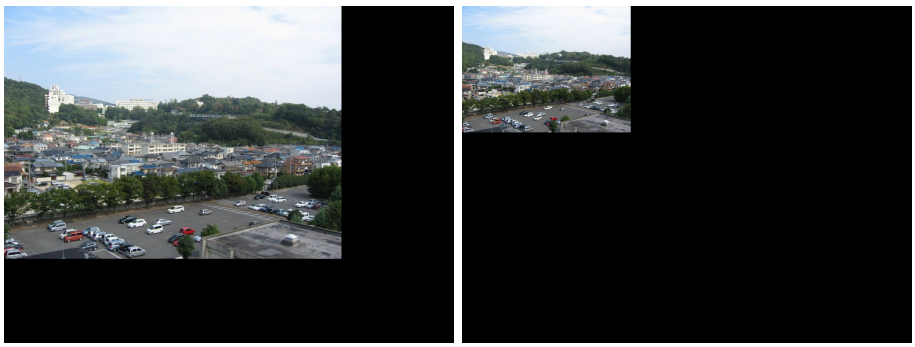


図 2　m が 1（左）と 0.5（右）のときの画像



図 3　t が 0（左）と $\frac{\pi}{4}$（右）のときの画像

1.2　[2-2]

1.2.1　m0d の変化による位置関係

m0d 関数を変更したとき，２つの画像の位置関係が変化しないことを確かめる．図 4 では以下のように変更をした．

```
1:    # 左の画像（もとの m0d 関数）
2:    m0d=np.array([[ 1,0,-100 ],
3:                  [ 0,1,-100 ],
4:                  [ 0,0,1 ]],'f')
5:    # 中央の画像（拡大させた）
6:    m0d=np.array([[ .5,0,-100 ],
7:                  [ 0,.5,-100 ],
8:                  [ 0,0,1 ]],'f')
9:    # 右の画像（平行移動をなくした）
10:   m0d=np.array([[ 1,0,0 ],
11:                 [ 0,1,0 ],
12:                 [ 0,0,1 ]],'f')
```



図 4　m0d 関数を変更した３つの画像

これらの画像の変化を比較したとき，拡大，平行移動させても 2 つの画像の位置関係は変化していないことがわかる．

1.2.2 ずれのない合成画像

次に m10 関数を変更し，ずれのない合成画像を生成する．m10 関数を以下のように変更したとき，生成される画像は図 5 のようになる．

```
1:    m10=np.array([’’’
2: 0.949961 0.007558 127.986108
3: -0.046213 0.996315 27.732034
4: -0.000113 -0.000012 1.040526 ’’’.split()]).astype(’f’).reshape(3,3)
```



図 5　ずれのない合成画像

2　感想

射影の理解が少しは深まった気がする．

参考文献